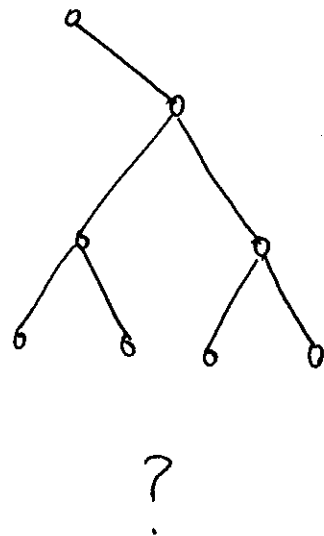
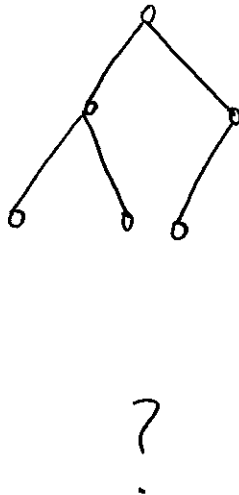
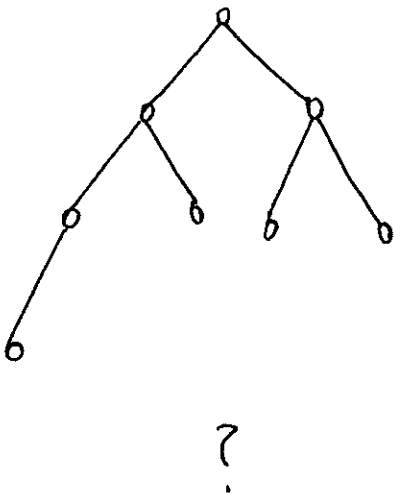
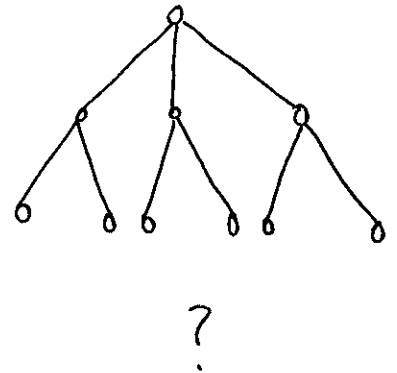
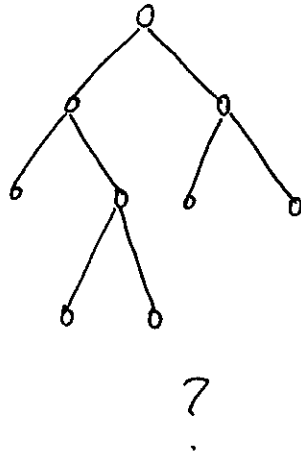
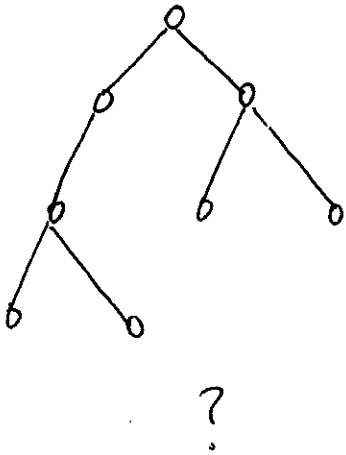


Heap Sort

A binary heap is a complete binary tree.

A complete binary tree is a binary tree such that except the last level, every level has been completely filled. For the last level, nodes are as far left as possible.

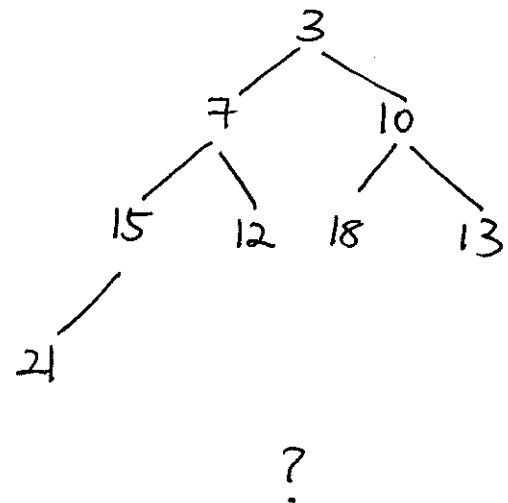
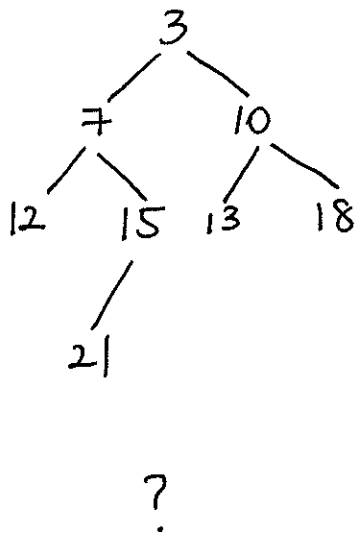
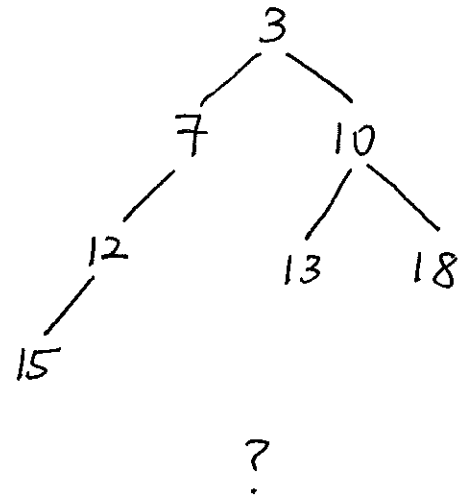
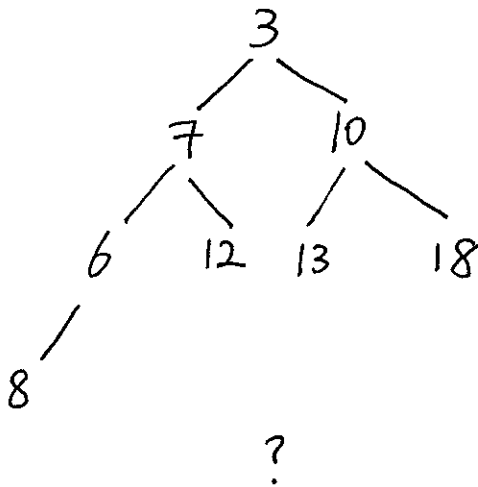


Maximum heap & Minimum heap

We will just cover minimum heap since max heap is just the opposite of min heap.

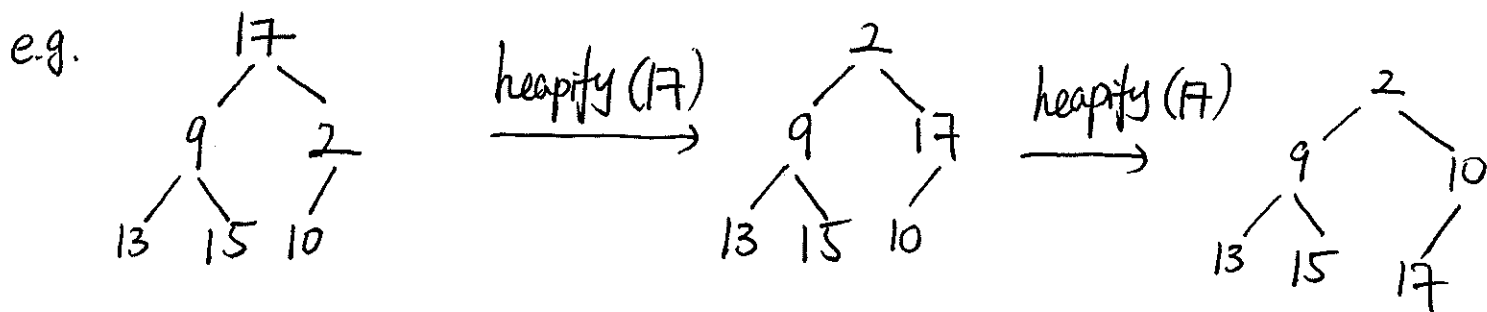
Definition: A minimum heap is a binary heap such that for any internal node t in the heap, the key of t is less than or equal to the keys of the children of t .
(Note: root has the min key of the heap)

exercise:



maximum heap: leave it to the readers

Heapify (min-heapify) a node t :



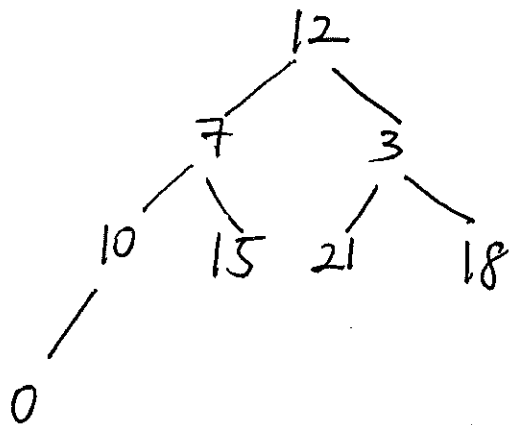
- 1° find the smallest key among t , t 's left and t 's right
- 2° If t is not the smallest, swap (t , smallest).

Then heapify t again.

How to build a min heap?

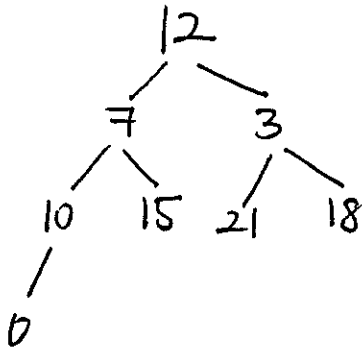
e.g. 12 7 3 10 15 21 18 0

step 1: create a complete binary tree by simply filling the tree using input keys (from left to right)

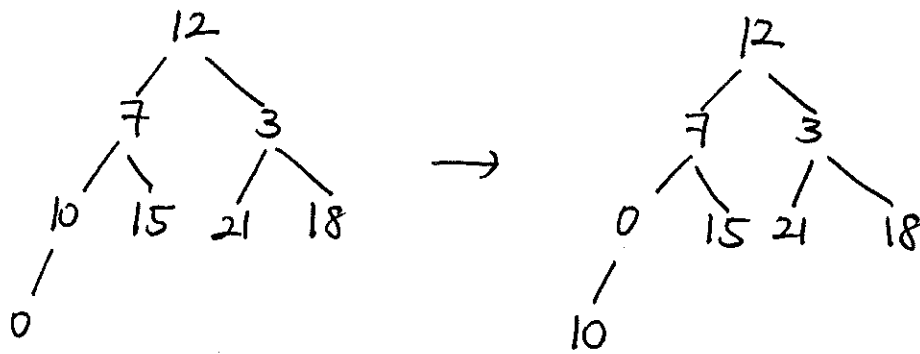


step 2: Heapify each internal node t from bottom up.
^{remember}
(If t is swapped, heapify t again.)
why

e.g.

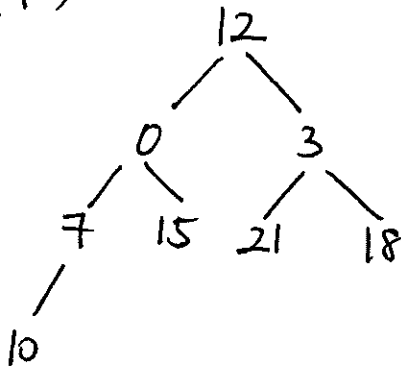


Heapify (10)



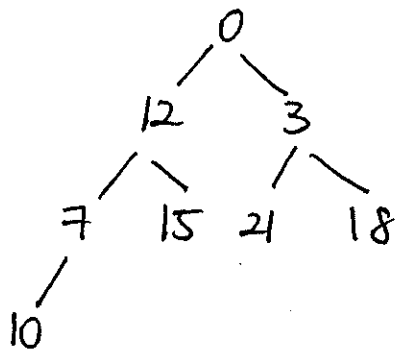
Heapify (3)

Heapify (7)

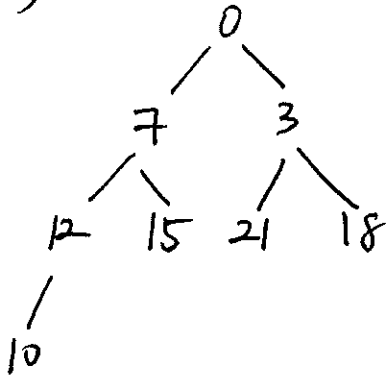


Heapify (7) again (why)

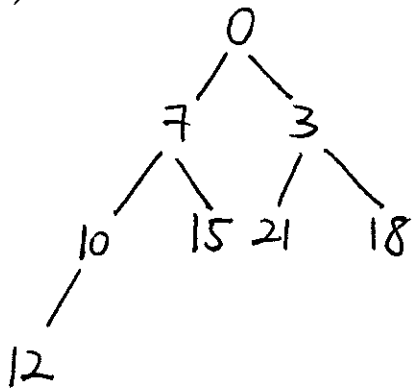
Heapify (12)



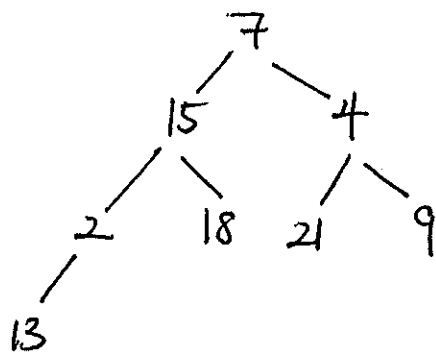
Heapify (12)



Heapify (12)



e.g. 7 15 4 2 18 21 9 13



Heapify (2)

Heapify (4)

Heapify (15) — swap (15, 2) — Heapify (15) — swap (15, 13)

Heapify (7) — swap (7, 2) — Heapify (7)

How to do Heap sort?

Step 1: build a min heap for the given input

(Note: root is the min of the tree)

Step 2: swap the root and the last element of the heap

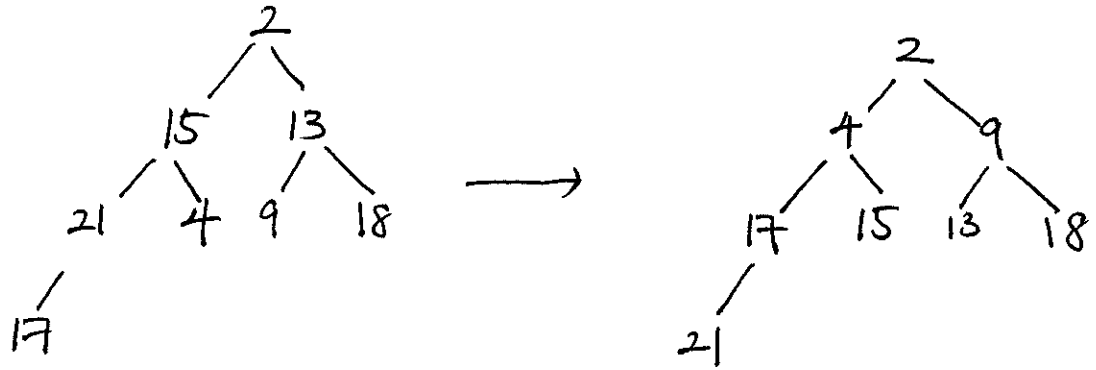
Step 3: delete (ignore) the last element in the tree

the heapify the root

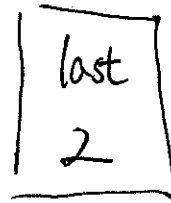
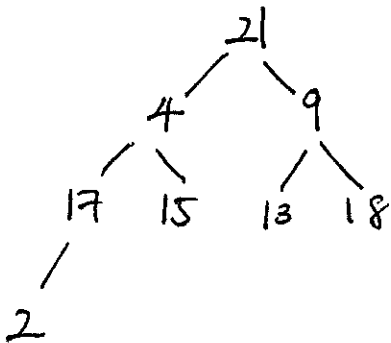
Step 4: repeat step 2 & 3 until heap size is 1.

e.g. 2 15 13 21 4 9 18 7

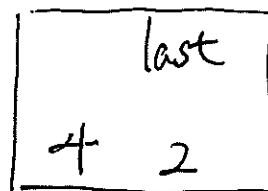
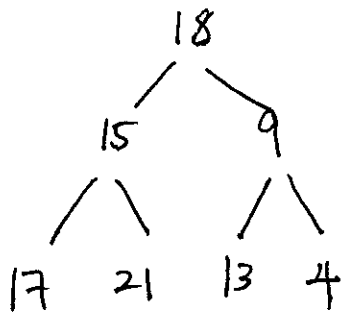
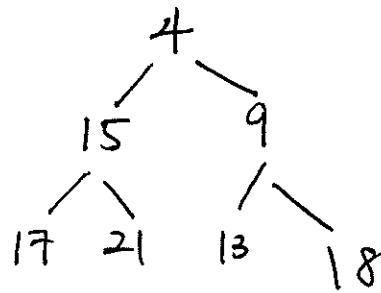
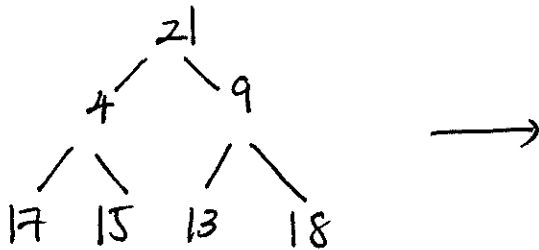
step 1:

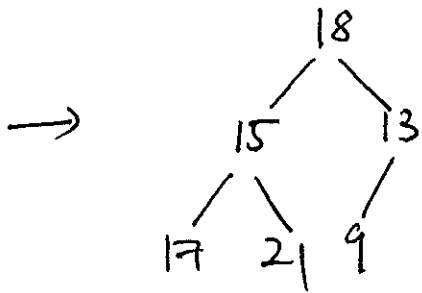
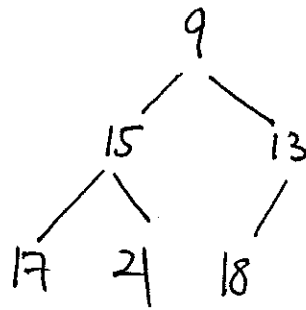
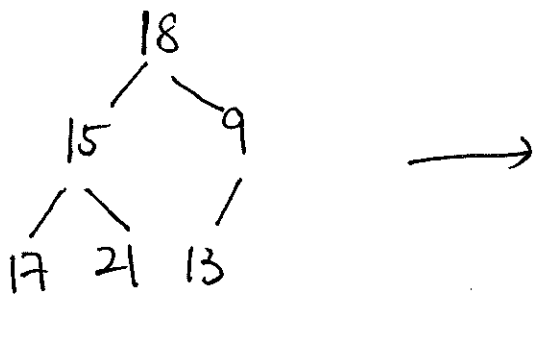


step 2:

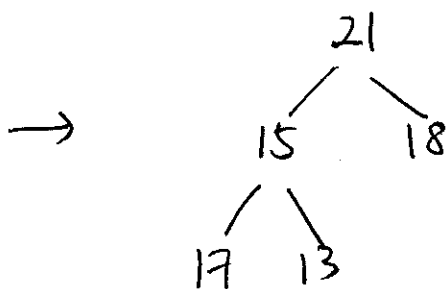
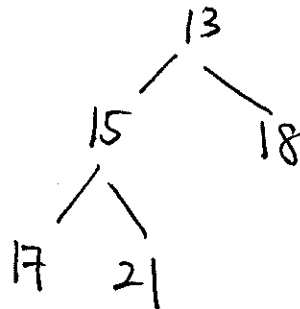
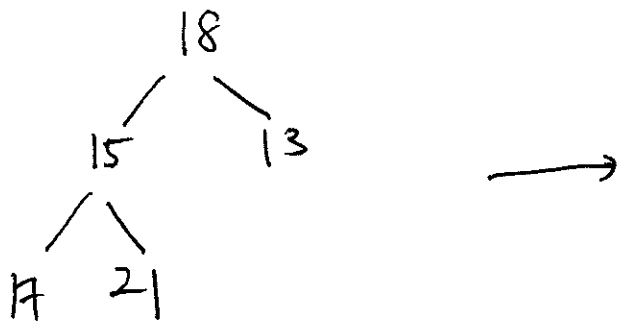


step 3:

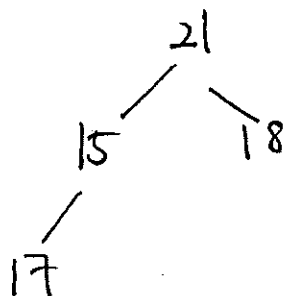




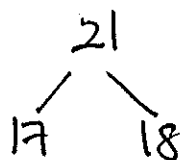
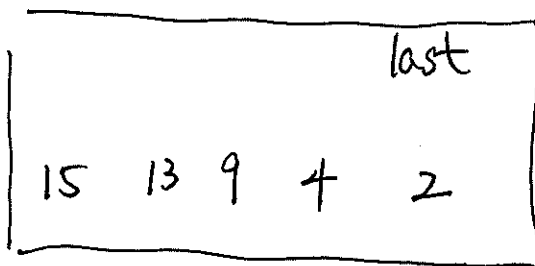
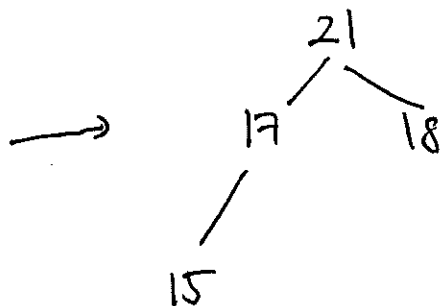
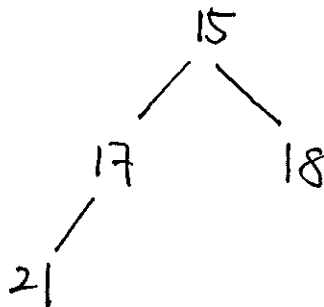
last		
9	4	2



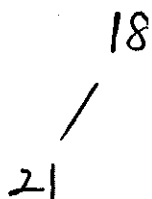
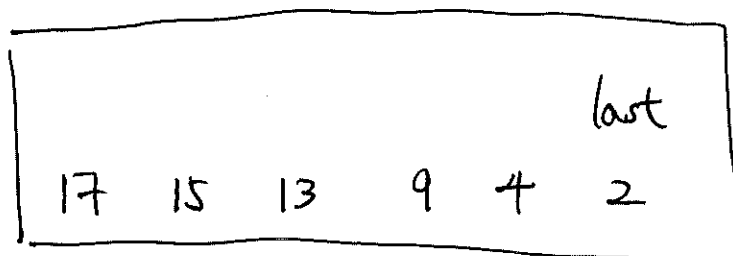
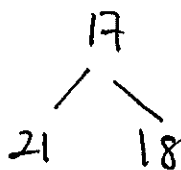
last			
13	9	4	2



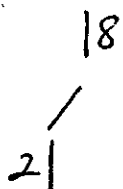
→



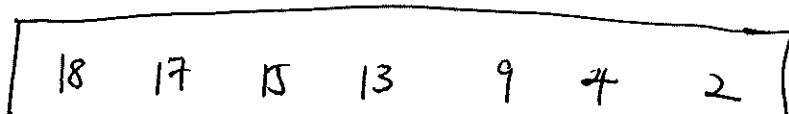
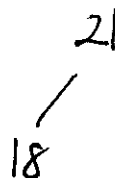
→



→



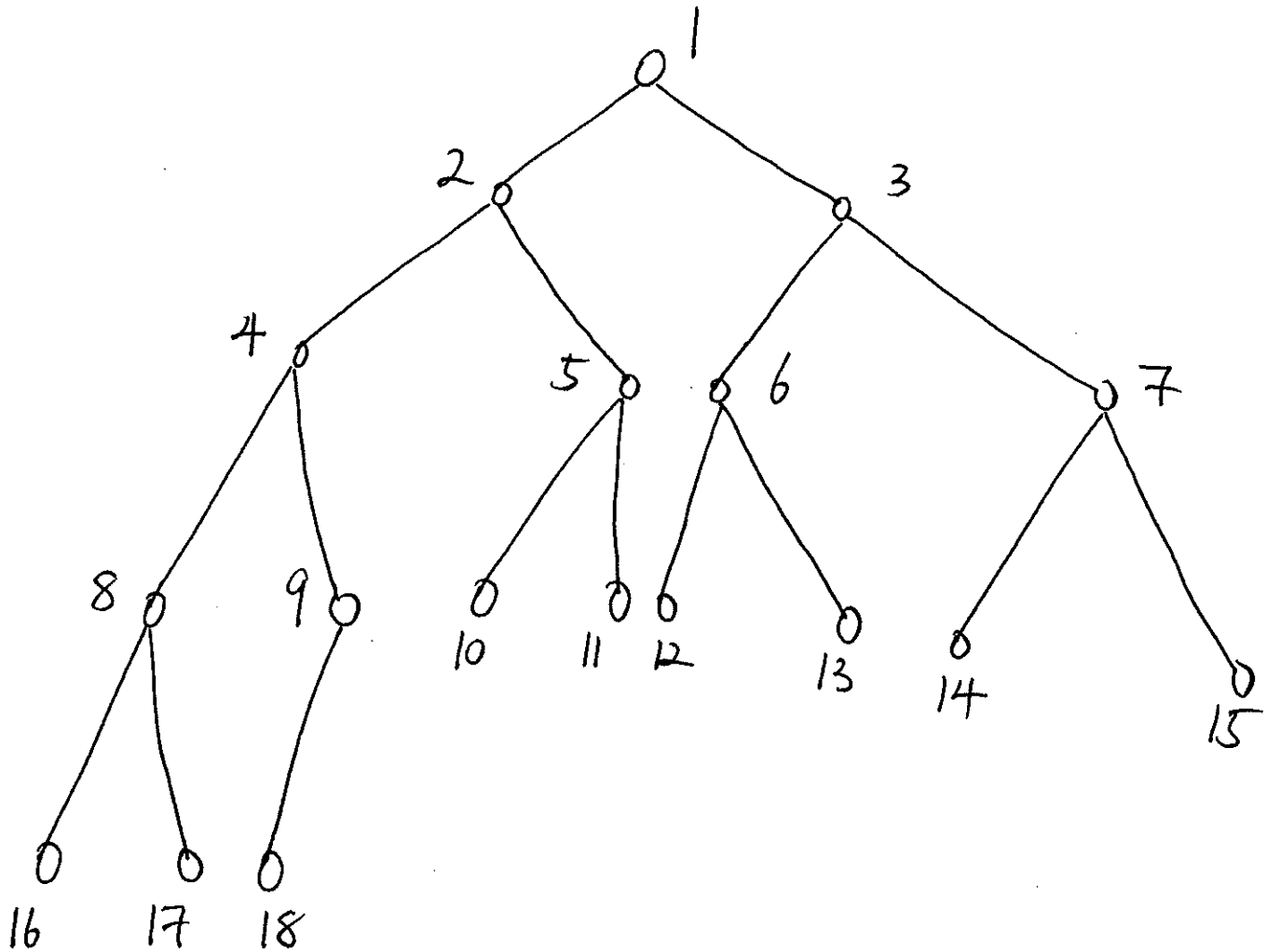
→



How to implement Heap sort ?

Which data structure can we use ?

array / vector



1's children : 2 3

2's children : 4 5

3's children : 6 7

4's children : 8 9

⋮

i's children : $2i$ $2i+1$

What are the indices of all internal nodes?

↑
We must heapify all internal nodes

e.g. 18 nodes : indices 1 to 9

9 nodes : 1 to 4

n nodes : 1 to $n/2$

heapifyNode (int i, int last)

int left = $2*i$, right = $2*i+1$;

int min = i;

find the min index first, then

do at most one swap (`std::swap(vect[i], vect[min])`)

Build a heap — `heapify (std::vector<int> sort, int last)`

for (int i = last/2 ; i >= 1 ; i--) {

 heapifyNode (i, last);

}

Time Complexity:

step 2-4

n nodes — Height: $O(\log n)$

heapify a node: $O(\log n)$

repeat heapify a node $O(n)$ times : $O(n \cdot \log n)$

step 1: Build a heap

$O(n)$ internal nodes and heapify each node

$\Rightarrow$ Build a heap : $O(n \cdot \log n)$

(There is a tighter bound $O(n)$)

Totally, step 1 + step 2-4

$$O(n \cdot \log n) + O(n \cdot \log n) = O(n \cdot \log n)$$

- Build a heap (max-heap)

idea: heapify every non-leaf

question: top-down OR bottom-up?

build-max-heap(A, n) {

$A.\text{heap-size} = n$;

 for $i = \lfloor n/2 \rfloor$ to 1 {

 max-heapify(A, i);

 }

}

☆☆☆ - Time Complexity for building a heap

- Keep in mind: the running time for building a heap is

$$c \cdot n = O(n)$$

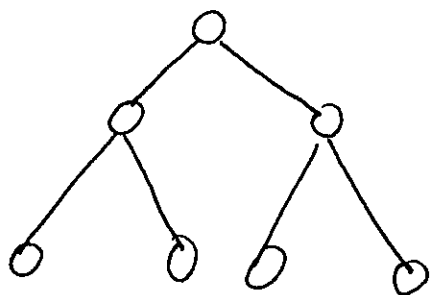
- Why? A little Math here:

① height of the heap: $\lfloor \log n \rfloor$

② # of nodes of any height h : $\lceil n/2^{h+1} \rceil$

e.g.

$n=7$



← $h=2$

$$\lceil 7/2^{2+1} \rceil = 1$$

← $h=1$

$$\lceil 7/2^{1+1} \rceil = 2$$

← $h=0$

$$\lceil 7/2^{0+1} \rceil = 4$$

③ heapify a node of height h : $c \cdot h = O(h)$

Time for building a heap

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot c h \leq \sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^h} \cdot c h$$

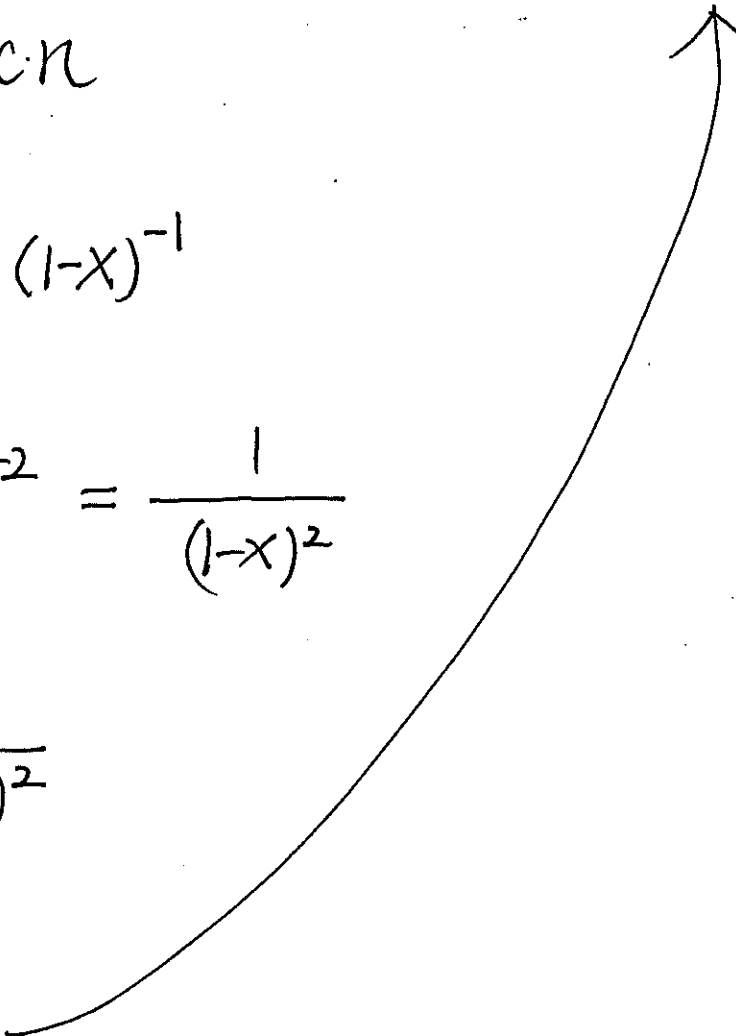
$$= c \cdot n \cdot \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \leq c \cdot n \cdot \sum_{h=0}^{\infty} \frac{h}{2^h}$$

$$= 2 \cdot c \cdot n$$

$$\sum_{h=0}^{\infty} x^h = \frac{1}{1-x} = (1-x)^{-1}$$

$$\Rightarrow \sum_{h=0}^{\infty} h \cdot x^{h-1} = (1-x)^{-2} = \frac{1}{(1-x)^2}$$

$$\Rightarrow \sum_{h=0}^{\infty} h \cdot x^h = \frac{x}{(1-x)^2}$$

$$\text{let } x = \frac{1}{2}$$


Priority Queue

A priority queue is an abstract data-type similar to a regular queue (FIFO), but each element in a priority queue has an associated priority.

- Elements with high priority are served before elements with low priority.
- e.g. hospital
- Priority queues are often implemented using heaps, so think about implementing the following standard operations using binary heaps. Function names assume a min heap.

* `find-min()`: returns the element with the highest priority
time complexity ?

* `extract-min()`: remove the element with the highest priority
time complexity $O(\log n)$
`heapify()` after removing the root

* `insert()`: insert a new element
add the new element at the end of the tree
traverse up to fix violated heap property
time complexity $O(\log n)$

* decrease-key(): decrease the value of the key, so the priority becomes higher.

If the decreased key value is greater than its parent, then do nothing. Otherwise, traverse up to fix violated heap property.

time complexity $O(\log n)$

- Priority Queues can also be implemented using Fibonacci heaps

	find-min	extract-min	insert	decrease-key
binary heap	$\theta(1)$	$\theta(\log n)$	$O(\log n)$	$O(\log n)$
Fibonacci heap	$\theta(1)$	$O(\log n)$	$\theta(1)$	$\theta(1)$

Radix Sort

Review:

The time complexity of

Quick sort in best case

merge sort

heap sort

The best performance of comparison based sorting algorithm is
 $O(n \cdot \log n)$

Radix sort is a non-comparison based sorting algorithm.
It avoids comparison by distributing elements into buckets according to their radix.

Radix sort considers every digit of each element in a certain order.

Digit Order

LSD: least significant digit

724

725 vs 824 vs 734

which one?

MSD: most significant digit

Strong

Which one is the MSD? Which one is the LSD?

Radix Sort

e.g. base 10 numbers, ascending order

170, 45, 75, 90, 2, 802, 2, 66

step 0: consider all elements to have the same number of digits

170, 045, 075, 090, 002, 802, 002, 066

step 1: According to the LSD of each element, distribute elements into buckets (queues)

0	1	2	3	4	5	6	7	8	9
170		002			045	066			
090		802			075				
		002							

170 090 002 802 002 045 075 066

step 2: According to the second LSD

0	1	2	3	4	5	6	7	8	9
002				045		066	170		090
802							075		
002									

002 802 002 045 066 170 075 090

step 3: consider the MSD

0	1	2	3	4	5	6	7	8	9
002	170							802	
002									
045									
066									
075									
090									

002 002 045 066 075 090 170 802

e.g.

BIG TEA TEAM BOB BE KEY COW GAT FOOT

step 0: make all elements have the same number of digits

BIG_ TEA_ TEAM BOB_ BE__ KEY_ COW_ GAT_ FOOT

step 1: What are the buckets we need?

What is the LSD of each element?

_ A B C M T X Y Z

BIG_ TEAM FOOT

TEA_

BOB_

BE__

KEY_

COW_

GAT_

BIG_ TEA_ BOB_ BE__ KEY_ COW_ GAT_ TEAM FOOT

step 2:

└─ A B G O T W X Y Z

~~BE~~ TEAM BOB BIG FOOT CAT COW KEY

BE TEAM

BE TEAM TEAM BOB BIG FOOT CAT COW KEY

step 3:

└─ A B C D E I O -

CAT

BE BIG

BOB

TEAM

FOOT

TEAM

COW

KEY

CAT BE TEAM TEAM KEY BIG BOB FOOT COW

step 4:

└─ A B C D E F K T -

BE CAT

FOOT

KEY

TEA

BIG COW

TEAM

BOB

BE BIG BOB CAT COW FOOT KEY TEA TEAM

Time Complexity :

Sorting n elements, each element has m digits

How many steps?

m

Each step, we go through n elements once.

$O(n)$

totally : $O(m \cdot n)$

Data structure to implement

each bucket is a queue

first in first out is key

How many buckets : fixed

```
std::vector<std::queue<int>>
```